LAPORAN TUGAS 3 DRAFT DESAIN AWAL PROJECT

TASK SCHEDULING PADA CLOUD WORKFLOW MENGGUNAKAN HETEROGENEOUS EARLIEST FINISH TIME (HEFT) UNTUK OPTIMASI MAKESPAN DAN KONSUMSI ENERGI



Disusun oleh:

Rayhan Agnan Kusuma - 5027241102

Ahmad Idza Anafin - 5027241017

Andi Naufal Zaki - 5027241059

Zidny Ilman Nafi'an - 5027221072

Dosen Pengampu:

Henning Titi Ciptaningtyas, S.Kom, M.Kom.

Institut Teknologi Sepuluh Nopember

Surabaya

2026

DAFTAR ISI

BAB I PENDAHULUAN1

1.1 Latar Belakang1

1.2 Rumusan Masalah2

1.3 Tujuan2

1.4 Batasan Masalah2

1.5 Manfaat3

1.6 Sistematika Penulisan3

BAB II LANDASAN TEORI5

2.1 Komputasi Awan5

2.2 Virtualisasi dan Penjadwalan Task5

2.3 Algoritma Heterogeneous Earliest Finish Time (HEFT)6

2.4 Simulator CloudSim dan WorkflowSim8

2.5 Metrik Kinerja Penjadwalan8

2.6 Dataset Beban Kerja Cloud9

BAB III DRAFT DESAIN AWAL PROJECT10

3.1 Gambaran Umum dan Alur Penelitian10

3.2 Jenis Task/Workload dan Dataset11

3.3 Desain Arsitektur Cloud13

3.4 Fungsi Obyektif15

3.5 Metrik Optimasi yang Diukur16

3.6 Batasan Cloud18

BAB IV RENCANA PENGUJIAN DAN ESTIMASI AWAL KINERJA19

4.1 Skenario Pengujian19

4.2 Estimasi Analitis Kinerja HEFT pada Skenario S120

4.3 Analisis Awal22

4.4 Ekspektasi Hasil Simulasi23

4.5 Catatan Teknis dan Risiko Implementasi24

BAB V PENUTUP25

5.1 Kesimpulan Sementara25

5.2 Rencana Tindak Lanjut25

DAFTAR PUSTAKA27

BAB I PENDAHULUAN

1.1 Latar Belakang

Komputasi awan (*cloud computing*) menyediakan sumber daya komputasi, penyimpanan, dan jaringan yang dapat diakses sesuai kebutuhan dan dibayar sesuai pemakaian [1], [2]. Kemampuan ini mendorong banyak aplikasi ilmiah dan industri dijalankan di lingkungan cloud, terutama aplikasi yang terdiri atas banyak tugas (*task*) yang saling bergantung. Aplikasi semacam ini lazim dimodelkan sebagai *workflow* berbentuk *Directed Acyclic Graph* (DAG), yaitu simpul mewakili task dan sisi mewakili ketergantungan data antar-task.

Pada lingkungan cloud, sebuah workflow dieksekusi di atas kumpulan mesin virtual (*virtual machine*, VM) yang dihosting pada server fisik dengan kapasitas yang berbeda-beda. Penjadwalan task (*task scheduling*), yaitu penentuan urutan dan VM tempat setiap task dijalankan, sangat menentukan waktu penyelesaian workflow dan efisiensi pemakaian sumber daya. Masalah penjadwalan task pada sumber daya yang heterogen termasuk masalah NP-complete [3], sehingga pendekatan heuristik yang menghasilkan solusi baik dalam waktu komputasi singkat banyak digunakan.

Heterogeneous Earliest Finish Time (HEFT) adalah salah satu algoritma *list scheduling* yang paling banyak dijadikan acuan untuk penjadwalan DAG pada sumber daya heterogen [4]. HEFT mengurutkan task berdasarkan prioritas *upward rank* yang memperhitungkan biaya komputasi dan komunikasi, kemudian menempatkan setiap task pada VM yang memberikan waktu selesai paling awal. Karakteristik tersebut sesuai dengan kondisi penelitian ini, yaitu workload berbentuk DAG dan sumber daya cloud yang heterogen.

Selain waktu penyelesaian, konsumsi energi pusat data menjadi perhatian penting karena berpengaruh langsung terhadap biaya operasional dan dampak lingkungan [10]. Waktu penyelesaian (*makespan*) dan konsumsi energi saling berkaitan: server fisik yang menyala tetap mengonsumsi daya meskipun sedang menganggur [11], sehingga penjadwalan yang mempersingkat makespan berpotensi menurunkan energi, tetapi pemilihan VM yang cepat juga dapat

meningkatkan daya yang terpakai. Oleh sebab itu kedua tujuan tersebut perlu dievaluasi bersama.

Laporan ini memuat draft desain awal project yang mencakup pemilihan workload dan dataset, rancangan arsitektur cloud yang akan disimulasikan, fungsi objektif, metrik optimasi, serta batasan cloud. Algoritma HEFT digunakan sebagai algoritma heuristik utama pada tahap implementasi awal dan menjadi *baseline* bagi pengembangan pendekatan penjadwalan berikutnya. Rancangan pada laporan ini akan digunakan kembali pada implementasi simulator di tahap berikutnya (Tugas 4).

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam laporan ini adalah sebagai berikut.

1. Workload dan dataset apa yang sesuai untuk menguji algoritma HEFT pada penjadwalan workflow di lingkungan cloud heterogen?
2. Bagaimana rancangan arsitektur cloud, meliputi spesifikasi dan jumlah *datacenter*, *host*, VM, serta task/*cloudlet*, yang akan disimulasikan?
3. Fungsi objektif apa yang dioptimalkan, dan bagaimana HEFT digunakan untuk menghasilkan pemetaan task ke VM yang dievaluasi terhadap fungsi objektif tersebut?
4. Metrik optimasi dan batasan cloud apa yang digunakan, serta bagaimana estimasi awal kinerja HEFT pada skenario pengujian yang dirancang?

1.3 Tujuan

Tujuan penyusunan draft desain awal project ini adalah sebagai berikut.

1. Menentukan jenis task/workload dan dataset yang akan diujicobakan.
2. Merancang arsitektur cloud heterogen beserta spesifikasi dan jumlah datacenter, host, VM, dan task/cloudlet.
3. Merumuskan minimal dua fungsi objektif, yaitu minimasi makespan dan minimasi konsumsi energi.

4. Menetapkan metrik optimasi yang diukur dan batasan cloud yang harus dipenuhi oleh penjadwal.
5. Menyusun rencana pengujian dan estimasi analitis awal kinerja HEFT sebagai bekal implementasi simulator.

1.4 Batasan Masalah

Agar penelitian terfokus, ruang lingkup dibatasi sebagai berikut.

1. Algoritma penjadwalan yang digunakan adalah HEFT sebagai algoritma heuristik utama dan baseline; perbandingan dengan algoritma lain tidak termasuk dalam ruang lingkup laporan ini.
2. Evaluasi dilakukan melalui simulasi menggunakan WorkflowSim yang berbasis CloudSim, bukan pada infrastruktur cloud nyata.
3. Workload berupa workflow DAG, yaitu DAG sintetis (100, 250, dan 500 task) serta workflow ilmiah Montage dan CyberShake.
4. Lingkungan cloud terdiri atas 2 datacenter heterogen dengan total 8 host fisik, dan jumlah VM 10, 20, atau 40.
5. Konsumsi energi dihitung hanya untuk host fisik dengan model daya linear; energi pendinginan dan perangkat jaringan tidak dimodelkan.
6. Task bersifat *non-preemptive*, tidak ada kegagalan task maupun VM, dan tidak ada migrasi VM selama eksekusi.
7. Bab IV berisi rencana pengujian dan estimasi analitis; hasil simulasi sesungguhnya akan disajikan pada tahap implementasi.

1.5 Manfaat

Manfaat yang diharapkan dari penyusunan laporan ini adalah sebagai berikut.

- **Manfaat akademis**: menyediakan rancangan eksperimen yang terdokumentasi, terukur, dan dapat direplikasi untuk penelitian penjadwalan workflow di cloud.
- **Manfaat teknis**: menjadi acuan konfigurasi simulator (WorkflowSim/CloudSim) sehingga implementasi pada tahap berikutnya dapat langsung dikerjakan tanpa perancangan ulang.

- **Manfaat pengembangan**: menghasilkan baseline HEFT yang dapat dibandingkan dengan pendekatan penjadwalan lain, termasuk pendekatan yang memperhitungkan energi secara eksplisit.

1.6 Sistematika Penulisan

Laporan ini disusun dalam lima bab dengan sistematika sebagai berikut.

- **BAB I Pendahuluan** berisi latar belakang, rumusan masalah, tujuan, batasan masalah, manfaat, dan sistematika penulisan.
- **BAB II Landasan Teori** membahas komputasi awan, virtualisasi dan penjadwalan task, algoritma HEFT, simulator CloudSim dan WorkflowSim, metrik kinerja penjadwalan, serta dataset beban kerja cloud.
- **BAB III Draft Desain Awal Project** menguraikan alur penelitian, jenis task/workload dan dataset, desain arsitektur cloud, fungsi obyektif, metrik optimasi, dan batasan cloud.
- **BAB IV Rencana Pengujian dan Estimasi Awal Kinerja** memuat skenario pengujian, estimasi analitis kinerja HEFT pada skenario S1, analisis awal, ekspektasi hasil simulasi, serta catatan teknis dan risiko implementasi.
- **BAB V Penutup** berisi kesimpulan sementara dan rencana tindak lanjut.

BAB II LANDASAN TEORI

2.1 Komputasi Awan

Menurut *National Institute of Standards and Technology* (NIST), komputasi awan adalah model yang memungkinkan akses jaringan yang mudah dan sesuai permintaan ke kumpulan sumber daya komputasi terkonfigurasi bersama (misalnya jaringan, server, penyimpanan, aplikasi, dan layanan) yang dapat disediakan dan dilepas dengan cepat dengan upaya manajemen minimal [1]. Model ini memiliki lima karakteristik utama, yaitu layanan mandiri sesuai permintaan, akses jaringan luas, penggabungan sumber daya (*resource pooling*), elastisitas yang cepat, dan layanan terukur [1].

Layanan cloud umumnya dikelompokkan menjadi *Infrastructure as a Service* (IaaS), *Platform as a Service* (PaaS), dan *Software as a Service* (SaaS). Penelitian ini berada pada tataran IaaS, yaitu pengguna menyewa VM dan menentukan bagaimana task workflow ditempatkan pada VM tersebut. Armbrust dkk. menekankan bahwa ilusi sumber daya yang tak terbatas dan pembayaran sesuai pemakaian merupakan aspek baru dari cloud, sekaligus menuntut pemanfaatan sumber daya yang efisien [2].

Pusat data cloud pada praktiknya bersifat heterogen karena server dibeli dan diganti secara bertahap sehingga terdapat perbedaan generasi prosesor, kapasitas memori, dan bandwidth. Heterogenitas ini membuat waktu eksekusi sebuah task berbeda-beda bergantung pada VM tempat ia dijalankan, dan inilah yang membuat keputusan penjadwalan menjadi bermakna. Di sisi lain, pusat data merupakan konsumen energi yang besar sehingga efisiensi energi menjadi salah satu tujuan optimasi yang penting [10].

2.2 Virtualisasi dan Penjadwalan Task

Virtualisasi memungkinkan satu server fisik (*host*) menjalankan beberapa VM yang terisolasi, masing-masing dengan alokasi CPU virtual (vCPU), memori, penyimpanan, dan bandwidth tersendiri. Pengelolaan sumber daya pada cloud dapat dipandang sebagai dua tingkat keputusan. Tingkat pertama adalah penempatan VM pada host (*VM allocation*), sedangkan tingkat kedua adalah

penjadwalan task pada VM (*task scheduling*). Penelitian ini berfokus pada tingkat kedua, sementara penempatan VM pada host menggunakan kebijakan bawaan simulator.

Sebuah workflow dimodelkan sebagai graf berarah tanpa siklus G = (V, E). Setiap simpul $n_i \in V$ adalah task dengan panjang komputasi L_i (dalam *million instructions*, MI), dan setiap sisi $(n_i, n_j) \in E$ menyatakan bahwa n_j hanya boleh dimulai setelah n_i selesai dan data hasilnya diterima. Data yang ditransfer antar-task menimbulkan biaya komunikasi $c_{i,j}$, yang bernilai nol apabila kedua task dijalankan pada VM yang sama. Task tanpa predesesor disebut *entry task*, sedangkan task tanpa suksesor disebut *exit task*.

Penjadwalan DAG pada sumber daya heterogen tergolong masalah NP-complete [3], sehingga pendekatan praktis mengandalkan heuristik. Secara garis besar, algoritma penjadwalan workflow dapat dikelompokkan menjadi *list scheduling* (misalnya HEFT), *clustering*, duplikasi task, dan metaheuristik (misalnya algoritma genetika dan optimasi koloni partikel). Heuristik list scheduling menonjol karena kompleksitasnya rendah dan kualitas solusinya baik, sehingga sering dijadikan baseline [4].

2.3 Algoritma Heterogeneous Earliest Finish Time (HEFT)

HEFT diusulkan oleh Topcuoglu dkk. sebagai algoritma *list scheduling* untuk DAG pada prosesor heterogen [4]. Dalam konteks cloud, prosesor tersebut dipetakan sebagai VM. HEFT bekerja dalam dua fase, yaitu fase prioritisasi task dan fase pemilihan VM.

2.3.1 Fase Prioritisasi Task

Setiap task diberi prioritas berupa *upward rank* yang dihitung secara rekursif dari *exit task* ke arah *entry task*:

$$rank_u(n_i) = \bar{w}_i + \max_{n_j \in succ(n_i)} \left(\bar{c}_{i,j} + rank_u(n_j) \right) \quad (2.1)$$

dengan $\bar{w}_i$ adalah rata-rata biaya komputasi task n_i pada seluruh VM, $\bar{c}_{i,j}$ adalah rata-rata biaya komunikasi dari n_i ke n_j, dan $succ(n_i)$ adalah himpunan suksesor

langsung n_i. Untuk *exit task*, nilai $rank_u$ sama dengan $\bar{w}$ task tersebut. Biaya komputasi task n_i pada VM v dihitung dari panjang task dan kecepatan VM:

$$w_{i,v} = \frac{L_i}{MIPS_v} \quad (2.2)$$

Task kemudian diurutkan menurut $rank_u$ secara menurun. Karena $rank_u$ suatu task selalu lebih besar daripada $rank_u$ seluruh suksesornya, urutan ini otomatis memenuhi batasan ketergantungan (topologis).

2.3.2 Fase Pemilihan VM

Task diambil satu per satu dari daftar terurut. Untuk setiap VM v_j, HEFT menghitung *Earliest Start Time* (EST) dan *Earliest Finish Time* (EFT):

$$EST(n_i, v_j) = max\left\{avail(v_j), \max_{n_m \in pred(n_i)} \left(AFT(n_m) + c_{m,i}\right)\right\} \quad (2.3)$$

$$EFT(n_i, v_j) = EST(n_i, v_j) + w_{i,j} \quad (2.4)$$

dengan $avail(v_j)$ adalah waktu saat VM v_j siap menjalankan task berikutnya, $AFT(n_m)$ adalah *Actual Finish Time* predesesor n_m, dan $pred(n_i)$ adalah himpunan predesesor langsung n_i. Task ditempatkan pada VM yang memberikan EFT terkecil. HEFT juga menerapkan kebijakan *insertion-based*, yaitu task dapat disisipkan pada celah waktu kosong (*idle slot*) di antara dua task yang sudah terjadwal pada suatu VM selama ketergantungan tetap terpenuhi [4].

Langkah-langkah HEFT dirangkum sebagai berikut.

1. Hitung biaya komputasi $w_{i,v}$ untuk setiap pasangan task–VM dan biaya komunikasi $c_{i,j}$ untuk setiap sisi DAG.
2. Hitung $rank_u$ seluruh task mulai dari *exit task* dengan persamaan (2.1).
3. Urutkan task berdasarkan $rank_u$ menurun ke dalam daftar prioritas.
4. Selama daftar prioritas belum kosong, ambil task dengan prioritas tertinggi.
5. Hitung EFT task tersebut pada setiap VM dengan persamaan (2.3) dan (2.4).

6. Tempatkan task pada VM dengan EFT terkecil, lalu perbarui waktu tersedia VM tersebut.

Kompleksitas waktu HEFT adalah O(e × p) untuk e sisi dan p prosesor, atau sekitar O(v^2 × p) pada graf padat dengan v task [4]. Kompleksitas yang rendah ini membuat waktu penjadwalan HEFT sangat kecil dibandingkan metaheuristik yang membutuhkan banyak iterasi evaluasi. Keterbatasan HEFT adalah keputusan yang bersifat *greedy* per task terhadap satu kriteria, yaitu waktu selesai. HEFT tidak mempertimbangkan energi maupun batas waktu (*deadline*) secara eksplisit sehingga konsumsi energi hanya dapat dievaluasi setelah jadwal terbentuk. Keterbatasan inilah yang menjadikan hasil HEFT layak dipakai sebagai baseline untuk pendekatan lanjutan.

2.4 Simulator CloudSim dan WorkflowSim

CloudSim adalah *toolkit* simulasi lingkungan cloud yang memodelkan *datacenter*, *host*, VM, dan *cloudlet* (unit pekerjaan), sekaligus kebijakan alokasi VM dan penjadwalan di dalamnya [5]. Simulasi dilakukan pada level model sehingga eksperimen berskala besar dapat diulang dengan biaya rendah dan hasil yang terkendali. CloudSim juga dilengkapi paket *power* yang memungkinkan pemodelan konsumsi daya host, misalnya dengan model daya linear terhadap utilisasi CPU [10].

WorkflowSim memperluas CloudSim untuk mensimulasikan workflow ilmiah dengan menambahkan lapisan *workflow planner*, *workflow engine*, *clustering engine*, dan *workflow scheduler*, serta kemampuan membaca berkas DAX (format XML workflow Pegasus) dan memodelkan *overhead* serta kegagalan [6]. WorkflowSim menyediakan beberapa algoritma perencanaan (*planning*) dan penjadwalan bawaan, termasuk implementasi HEFT yang dapat dijadikan acuan validasi. Padanan konsep desain penelitian dengan komponen simulator ditunjukkan pada Tabel 2.1.

Tabel 2.1 Padanan konsep penelitian dengan komponen simulator

Konsep penelitian	Komponen CloudSim/WorkflowSim	Peran dalam eksperimen
Datacenter	Datacenter (PowerDatacenter)	Menampung host heterogen dan karakteristik biaya/jaringan

Konsep penelitian	Komponen CloudSim/WorkflowSim	Peran dalam eksperimen
Host fisik	Host (PowerHost) berisi PE	Menyediakan CPU (MIPS), RAM, bandwidth; dasar hitung energi
VM	Vm + VmScheduler	Unit tempat task dieksekusi; alokasi ke host
Task workflow	Cloudlet (Task) dari berkas DAX	Unit pekerjaan dengan parent, child, dan file data
Pemetaan task–VM	Planning/Scheduling algorithm (HEFT)	Menghasilkan pemetaan task ke VM
Eksekusi & ketergantungan	WorkflowEngine, WorkflowScheduler	Melepas task hanya bila semua parent selesai

2.5 Metrik Kinerja Penjadwalan

Kinerja algoritma penjadwalan perlu diukur dengan metrik yang mencerminkan kepentingan pengguna dan penyedia layanan. Dari sisi pengguna, metrik utama adalah *makespan*, yaitu waktu dari task pertama dimulai hingga task terakhir selesai. Dari sisi penyedia, metrik yang relevan adalah konsumsi energi dan tingkat pemanfaatan sumber daya (*resource utilization*). Utilisasi yang rendah menunjukkan sumber daya yang sudah dibayar dan dinyalakan tetapi tidak dimanfaatkan, sedangkan konsumsi energi mencerminkan biaya operasional dan dampak lingkungan.

Selain itu, waktu yang diperlukan algoritma untuk menghasilkan jadwal (*scheduling runtime*) mencerminkan biaya komputasi algoritma itu sendiri (*overhead*), yang menjadi penting ketika ukuran workflow membesar. Dalam literatur penjadwalan DAG, hasil makespan juga sering dinormalkan terhadap batas bawah panjang lintasan kritis menggunakan *Schedule Length Ratio* (SLR) agar dapat dibandingkan lintas workflow dengan ukuran berbeda [4]. Definisi formal setiap metrik disajikan pada Subbab 3.5.

2.6 Dataset Beban Kerja Cloud

Beban kerja untuk evaluasi penjadwalan dapat berasal dari jejak eksekusi nyata (*trace*) atau dari workflow yang dibangkitkan. Untuk penjadwalan yang memperhitungkan ketergantungan antar-task, diperlukan beban kerja yang secara

eksplisit menyatakan struktur DAG, ukuran komputasi task, dan volume data antar-task. Repositori workflow ilmiah dari proyek Pegasus menyediakan struktur workflow beserta generator dengan berbagai ukuran, sehingga banyak digunakan sebagai *benchmark* [7], [9].

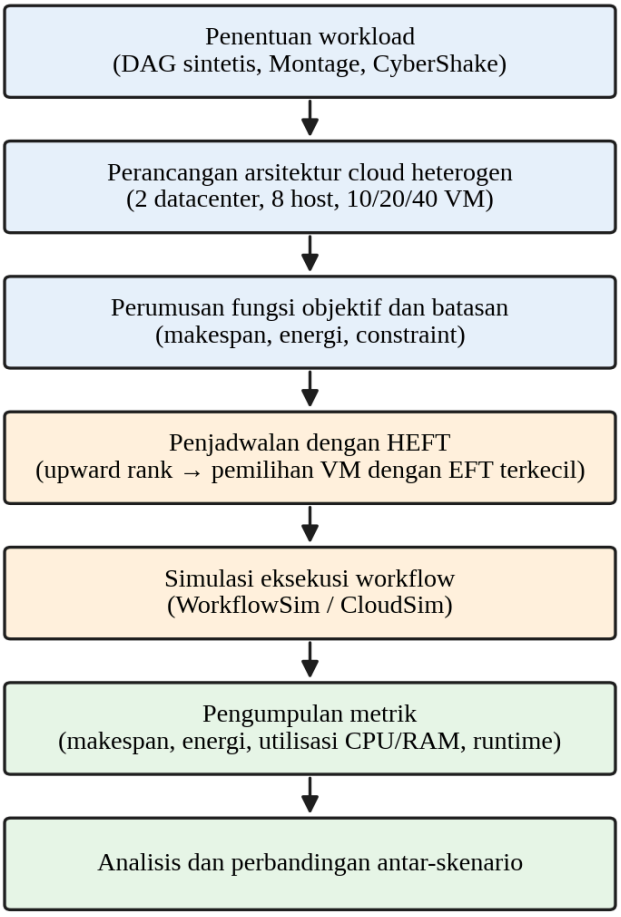
Dua workflow yang dipilih dalam penelitian ini memiliki karakteristik yang berbeda. **Montage** adalah workflow astronomi untuk menyusun mosaik citra langit dari banyak citra masukan; strukturnya banyak memiliki tahap paralel lebar (reproyeksi dan pengurangan latar) yang disusul tahap penggabungan, dan secara umum dikarakterisasi sebagai workflow yang berat pada operasi I/O [8]. **CyberShake** adalah workflow seismologi untuk analisis bahaya gempa probabilistik; workflow ini dikarakterisasi sebagai *data-intensive* dengan kebutuhan memori dan CPU yang besar pada sebagian task [8]. Perbedaan karakteristik tersebut memungkinkan evaluasi algoritma pada struktur ketergantungan yang lebih realistis.

Selain workflow nyata, workload sintetis dibangkitkan agar ukuran workflow, kepadatan ketergantungan, dan rasio komunikasi terhadap komputasi (*Communication-to-Computation Ratio*, CCR) dapat dikendalikan secara sistematis. Kombinasi keduanya memberikan keseimbangan antara keterkendalian eksperimen dan keterwakilan terhadap kasus nyata.

BAB III DRAFT DESAIN AWAL PROJECT

3.1 Gambaran Umum dan Alur Penelitian

Penelitian ini merancang sebuah lingkungan simulasi cloud heterogen tempat workflow DAG dijadwalkan menggunakan HEFT, kemudian hasil penjadwalan dievaluasi berdasarkan dua fungsi objektif, yaitu makespan dan konsumsi energi. Alur penelitian ditunjukkan pada Gambar 3.1. Tahap perancangan (workload, arsitektur, objektif dan batasan) diselesaikan pada laporan ini, sedangkan tahap penjadwalan, simulasi, dan analisis hasil dilaksanakan pada tahap implementasi.



Gambar 3.1 Alur penelitian

Ringkasan rancangan awal project ditunjukkan pada Tabel 3.1. Rincian setiap komponen dijelaskan pada subbab berikutnya.

Tabel 3.1 Ringkasan rancangan awal project

Komponen	Spesifikasi
Simulator	WorkflowSim berbasis CloudSim
Algoritma penjadwalan	HEFT (baseline dan algoritma utama pada implementasi awal)
Workload sintetis	DAG dengan 100, 250, dan 500 task
Workflow dunia nyata	Montage dan CyberShake
Datacenter	2 datacenter heterogen
Host fisik	8 host (4 host per datacenter), heterogen
Virtual machine	10, 20, dan 40 VM; 1–4 vCPU dan 2–8 GB RAM per VM
Fungsi objektif	Minimasi makespan dan minimasi konsumsi energi
Metrik	Makespan, energi, utilisasi CPU dan RAM, scheduling runtime
Batasan	Ketergantungan, kapasitas sumber daya, ketersediaan VM, deadline (opsional)

3.2 Jenis Task/Workload dan Dataset

Penelitian menggunakan workload berbasis workflow DAG. Setiap workflow terdiri atas sejumlah task yang memiliki hubungan ketergantungan sehingga sebuah task hanya dapat dijalankan setelah seluruh task pendahulunya selesai. Untuk menguji kinerja algoritma pada berbagai karakteristik workload, digunakan dua jenis workload, yaitu workload sintetis dan workload dunia nyata.

3.2.1 Workload Sintetis

Workload sintetis dibangkitkan dalam bentuk DAG berlapis (*layered*) dengan tiga variasi jumlah task, yaitu 100, 250, dan 500 task. Setiap task memiliki kebutuhan komputasi berupa panjang instruksi (*instruction length*) yang berbeda, sedangkan sisi antar-task merepresentasikan ketergantungan sekaligus biaya komunikasi. Variasi jumlah task digunakan untuk menguji skalabilitas algoritma ketika ukuran workflow meningkat. Parameter pembangkitan yang direncanakan ditunjukkan pada Tabel 3.2.

Tabel 3.2 Parameter pembangkitan workload sintetis

Parameter	Nilai rencana
Jumlah task (n)	100, 250, 500
Jumlah level DAG	$\approx \sqrt{n}$ (sekitar 10, 16, dan 22 level)

Parameter	Nilai rencana
Kepadatan sisi	≈ 1,8 sisi per task
Panjang task (L)	Seragam acak 1.000–10.000 MI (rata-rata 5.500 MI)
Jumlah PE per task	1 PE
Kebutuhan RAM per task	256–1.024 MB
Ukuran data per sisi	Seragam acak 10–50 MB
CCR	≈ 0,5
Entry/exit task	Satu entry dan satu exit (dummy, biaya nol) bila diperlukan
Pengulangan	10 DAG berbeda (seed acak) untuk setiap ukuran n

3.2.2 Workload Dunia Nyata

Workload dunia nyata menggunakan workflow ilmiah dari repositori workflow standar, yaitu Montage dan CyberShake. Kedua workflow dinyatakan sebagai DAG (berkas DAX) sehingga dapat langsung dibaca oleh WorkflowSim. Ukuran yang direncanakan adalah varian sekitar 100 task (Montage_100 dan CyberShake_100) yang tersedia pada Pegasus Workflow Generator [7]; ukuran dapat disesuaikan dengan berkas yang tersedia pada saat implementasi. Karakteristik kedua workflow diringkas pada Tabel 3.3.

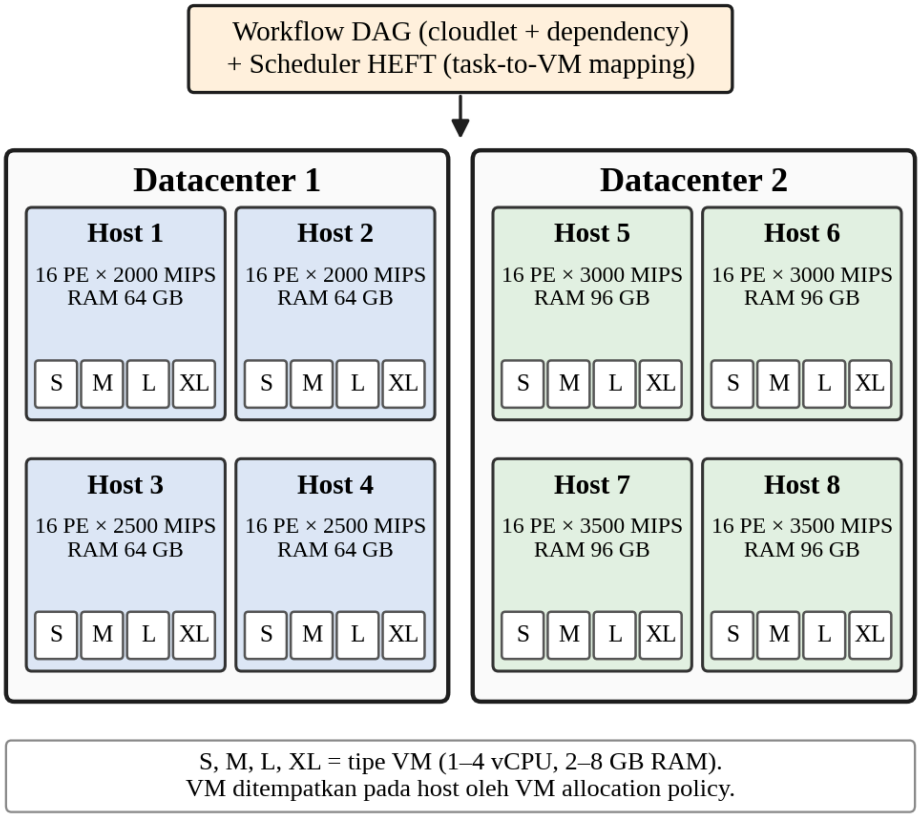
Tabel 3.3 Workflow dunia nyata yang digunakan

Workflow	Domain dan fungsi	Karakteristik utama [8]	Ukuran rencana
Montage	Astronomi; pemrosesan dan penyusunan mosaik citra langit	Tahap paralel lebar dengan penggabungan; berat pada I/O	Sesuai dataset (≈ 100 task)
CyberShake	Seismologi; simulasi dan analisis bahaya gempa bumi	*Data-intensive*; sebagian task berkebutuhan memori dan CPU besar	Sesuai dataset (≈ 100 task)

Dengan memakai workload sintetis dan workflow dunia nyata, eksperimen dapat mengevaluasi kinerja algoritma pada workload dengan ukuran dan struktur ketergantungan yang berbeda. Workload sintetis memberikan kendali atas ukuran dan CCR, sedangkan workflow nyata memberikan struktur ketergantungan yang lebih realistis.

3.3 Desain Arsitektur Cloud

Lingkungan cloud dibangun pada WorkflowSim berbasis CloudSim dengan sumber daya heterogen. Arsitektur terdiri atas dua datacenter, delapan host fisik, VM dalam tiga skenario jumlah, dan task/cloudlet yang berasal dari workflow DAG. Gambaran arsitektur ditunjukkan pada Gambar 3.2 dan ringkasan spesifikasi pada Tabel 3.4.



Gambar 3.2 Arsitektur cloud heterogen yang dirancang

Tabel 3.4 Ringkasan spesifikasi arsitektur cloud

Komponen	Spesifikasi
Datacenter	2 datacenter heterogen
Physical host	8 host, masing-masing 4 host per datacenter
VM	10, 20, dan 40 VM
vCPU	1–4 vCPU per VM
RAM	2–8 GB per VM
Workload	100–500 task untuk DAG sintetis; Montage dan CyberShake
Simulator	WorkflowSim / CloudSim

3.3.1 Datacenter

Digunakan dua datacenter yang merepresentasikan lingkungan cloud dengan sumber daya heterogen. Masing-masing datacenter memiliki empat host fisik. Datacenter 1 diisi host dengan kecepatan prosesor lebih rendah, sedangkan Datacenter 2 diisi host dengan kecepatan lebih tinggi dan memori lebih besar. Komunikasi antar-VM dalam satu datacenter dimodelkan dengan bandwidth 200 Mbps dan antar-datacenter 100 Mbps, sehingga penempatan task memengaruhi biaya komunikasi.

3.3.2 Physical Host

Total terdapat delapan host fisik yang dibuat heterogen dalam kapasitas komputasi dan memori. Spesifikasi setiap host ditunjukkan pada Tabel 3.5. Konsumsi daya host dimodelkan secara linear terhadap utilisasi CPU [10], [11], dengan daya saat menganggur (P_{idle}) dan daya maksimum (P_{max}). Nilai daya pada tabel merupakan nilai asumsi yang merepresentasikan server kelas menengah dan dapat disesuaikan pada saat implementasi.

Tabel 3.5 Spesifikasi host fisik

Host	Datacenter	PE	MIPS/PE	RAM	Bandwidth	P_{idle}	P_{max}
H1	DC1	16	2.000	64 GB	10 Gbps	100 W	250 W
H2	DC1	16	2.000	64 GB	10 Gbps	100 W	250 W
H3	DC1	16	2.500	64 GB	10 Gbps	110 W	270 W
H4	DC1	16	2.500	64 GB	10 Gbps	110 W	270 W
H5	DC2	16	3.000	96 GB	10 Gbps	130 W	300 W
H6	DC2	16	3.000	96 GB	10 Gbps	130 W	300 W
H7	DC2	16	3.500	96 GB	10 Gbps	150 W	350 W
H8	DC2	16	3.500	96 GB	10 Gbps	150 W	350 W

Catatan: penyimpanan tiap host 1 TB. Total kapasitas 128 PE dan 640 GB RAM. Jumlah P_{idle} seluruh host = 980 W.

3.3.3 Virtual Machine

VM dibuat dalam tiga skenario jumlah, yaitu 10, 20, dan 40 VM. Setiap VM memiliki konfigurasi berbeda dengan rentang 1–4 vCPU dan 2–8 GB RAM. Terdapat empat tipe VM (Tabel 3.6). Komposisi tipe VM pada setiap skenario

ditunjukkan pada Tabel 3.7, dan VM disebar merata pada kedua datacenter oleh kebijakan alokasi VM.

Tabel 3.6 Tipe VM

Tipe	vCPU	RAM	MIPS per vCPU	Kapasitas total (MIPS)
S	1	2 GB	1.000	1.000
M	2	4 GB	1.250	2.500
L	3	6 GB	1.500	4.500
XL	4	8 GB	2.000	8.000

Tabel 3.7 Komposisi VM pada setiap skenario

Skenario	S	M	L	XL	Total vCPU	Total kapasitas (MIPS)
10 VM	2	3	3	2	25	39.000
20 VM	5	5	5	5	50	90.000
40 VM	10	10	10	10	100	180.000

Catatan: pada skenario 40 VM dibutuhkan 100 vCPU dan 200 GB RAM, masih di bawah kapasitas host (128 PE dan 640 GB) sehingga tidak terjadi *over-allocation*.

3.3.4 Task / Cloudlet

Task direpresentasikan sebagai cloudlet dalam WorkflowSim. Task berasal dari workflow DAG sehingga memiliki beban komputasi (panjang dalam MI), task induk (*parent*), task anak (*child*), ketergantungan, dan biaya komunikasi. Setiap task membutuhkan 1 PE dan sejumlah RAM. Task hanya dapat dieksekusi apabila seluruh ketergantungan yang diperlukan telah terpenuhi. Jumlah cloudlet adalah 100, 250, atau 500 untuk DAG sintetis dan sekitar 100 untuk masing-masing workflow Montage dan CyberShake.

Kebijakan simulator yang direncanakan adalah *space-shared* pada tingkat penjadwal cloudlet dalam VM, sehingga satu vCPU hanya melayani satu task pada satu waktu dan eksekusi bersifat *non-preemptive*. Penempatan VM pada host menggunakan kebijakan alokasi bawaan CloudSim.

3.4 Fungsi Obyektif

Penelitian mengoptimalkan dan mengevaluasi dua tujuan utama, yaitu minimasi makespan dan minimasi konsumsi energi.

3.4.1 Objektif 1: Minimasi Makespan

Makespan adalah total waktu yang diperlukan untuk menyelesaikan seluruh workflow, yaitu waktu selesai terbesar dari seluruh task:

$$F_1 = C_{max} = \max_{n_i \in V} AFT(n_i) \quad (3.1)$$

Semakin kecil nilai makespan, semakin cepat seluruh workflow diselesaikan.

3.4.2 Objektif 2: Minimasi Konsumsi Energi

Konsumsi energi adalah total energi yang digunakan oleh seluruh host fisik selama workflow dieksekusi:

$$F_2 = E_{total} = \sum_{h=1}^{H} \int_{0}^{C_{max}} P_h(t)\,dt \quad (3.2)$$

dengan H jumlah host fisik (H = 8), $P_h(t)$ daya host h pada waktu t, dan C_{max} makespan workflow. Daya host dimodelkan linear terhadap utilisasi CPU host $U_h(t)$ [10], [11]:

$$P_h(t) = P_{idle,h} + \left(P_{max,h} - P_{idle,h}\right) \cdot U_h(t) \quad (3.3)$$

Dengan demikian, penelitian mempertimbangkan dua tujuan sekaligus, yaitu min (C_{max}, E_{total}). HEFT digunakan untuk menghasilkan pemetaan task ke VM, kemudian hasil penjadwalan dievaluasi berdasarkan kedua objektif tersebut.

Perlu dicatat bahwa HEFT pada dasarnya mengoptimalkan satu kriteria, yaitu waktu selesai. Energi tidak menjadi bagian dari keputusan HEFT, tetapi diukur sebagai konsekuensi jadwal yang dihasilkan. Karena host tetap mengonsumsi daya idle selama makespan berlangsung, kedua objektif tidak sepenuhnya independen: penurunan C_{max} cenderung menurunkan komponen energi idle. Hubungan ini dianalisis lebih lanjut pada Subbab 4.2 dan 4.3.

3.5 Metrik Optimasi yang Diukur

Kinerja penjadwalan dievaluasi menggunakan metrik yang dirangkum pada Tabel 3.8 dan didefinisikan sebagai berikut.

Tabel 3.8 Metrik optimasi yang diukur

Metrik	Definisi singkat	Satuan	Arah yang diinginkan
Makespan	Waktu penyelesaian seluruh workflow	detik	Lebih kecil
Energy consumption	Total energi seluruh host fisik	J atau kWh	Lebih kecil
CPU utilization	Persentase kapasitas vCPU yang dipakai eksekusi task	%	Lebih besar
RAM utilization	Persentase RAM VM yang dipakai task selama eksekusi	%	Lebih besar
Scheduling runtime	Waktu komputasi HEFT untuk menghasilkan jadwal	ms	Lebih kecil

3.5.1 Makespan

Makespan mengukur total waktu penyelesaian workflow, yaitu Makespan = C_{max} sebagaimana pada persamaan (3.1).

3.5.2 Energy Consumption

Energi mengukur total konsumsi energi host fisik selama simulasi, dihitung dengan persamaan (3.2), dan dilaporkan dalam Joule atau kWh (1 kWh = $3{,}6 \times 10^6$ J).

3.5.3 Resource Utilization

Utilisasi mengukur seberapa efektif sumber daya cloud digunakan oleh hasil penjadwalan, terutama CPU dan RAM:

$$U_{CPU} = \frac{t_i^{exec}}{vcpu_v \cdot C_{max}} \quad (3.4)$$

$$U_{RAM} = \frac{r_i \cdot t_i^{exec}}{RAM_v \cdot C_{max}} \quad (3.5)$$

dengan t_i^{exec} waktu eksekusi task n_i, r_i kebutuhan RAM task n_i, $vcpu_v$ jumlah vCPU VM v, dan RAM_v kapasitas RAM VM v. Metrik ini digunakan untuk melihat seberapa efektif sumber daya cloud dimanfaatkan.

3.5.4 Scheduling Runtime

Scheduling runtime mengukur waktu komputasi yang diperlukan algoritma HEFT untuk menghasilkan keputusan penjadwalan (penghitungan rank hingga penempatan seluruh task), diukur dengan penghitung waktu sistem pada program simulator. Metrik ini digunakan untuk mengevaluasi *computational overhead* algoritma.

3.5.5 Metrik Pendukung

Sebagai metrik pendukung, *Schedule Length Ratio* (SLR) dapat digunakan untuk menormalkan makespan terhadap batas bawah lintasan kritis [4]:

$$SLR = \frac{C_{max}}{\min_{v} w_{i,v}} \quad (3.6)$$

dengan CP_{MIN} adalah lintasan kritis DAG bila setiap task dihitung menggunakan biaya komputasi minimum tanpa biaya komunikasi. Nilai SLR selalu lebih besar atau sama dengan 1, dan nilai yang lebih kecil menunjukkan jadwal yang lebih baik.

3.6 Batasan Cloud

Penjadwalan harus memenuhi batasan (*constraint*) berikut.

3.6.1 Dependency Constraint

Task anak tidak boleh dimulai sebelum seluruh task induk selesai dan data hasilnya diterima. Untuk ketergantungan $T_i \rightarrow T_j$ berlaku:

$$Start(T_j) \geq Finish(T_i) + c_{i,j} \quad (3.7)$$

3.6.2 Resource Capacity Constraint

Task hanya dapat ditempatkan pada VM apabila sumber daya yang tersedia mencukupi kebutuhan task. Sumber daya yang diperhatikan meliputi CPU, RAM, dan bandwidth. Tidak diperbolehkan terjadi *resource over-allocation*. Pada setiap waktu t dan setiap VM v, dengan $R_v(t)$ himpunan task yang sedang berjalan pada v:

$$pe_i \leq vcpu_v, \quad r_i \leq RAM_v \quad (3.8)$$

dengan pe_i = 1 PE per task. Pada tingkat host, total vCPU dan RAM seluruh VM yang dialokasikan tidak boleh melebihi kapasitas PE dan RAM host.

3.6.3 VM Availability Constraint

Task harus dijadwalkan pada VM yang tersedia. Waktu eksekusi task harus memperhatikan saat VM menjadi tersedia setelah menjalankan task sebelumnya; dalam HEFT hal ini dinyatakan oleh avail(v_j) pada persamaan (2.3). Karena eksekusi bersifat *non-preemptive*, task yang sudah berjalan tidak dapat dihentikan atau dipindahkan.

3.6.4 Workflow Deadline

Apabila deadline digunakan dalam skenario eksperimen, seluruh workflow harus diselesaikan sebelum deadline D yang ditentukan:

$$C_{max} \leq D \quad (3.9)$$

Deadline digunakan sebagai batasan tambahan (opsional) untuk menguji kemampuan penjadwal memenuhi batas waktu penyelesaian workflow. Nilai D direncanakan sebagai $D = \beta \times CP_{MIN}$ dengan β = 1,5 dan 2,0, sehingga deadline mengikuti ukuran dan struktur masing-masing workflow.

BAB IV RENCANA PENGUJIAN DAN ESTIMASI AWAL KINERJA

4.1 Skenario Pengujian

Eksperimen dilakukan menggunakan kombinasi lima workload dan tiga jumlah VM sehingga menghasilkan 15 skenario (S1–S15), sebagaimana ditunjukkan pada Tabel 4.1. Untuk setiap skenario, simulator menjalankan workflow menggunakan algoritma HEFT.

Tabel 4.1 Skenario pengujian

Skenario	Workload	Jumlah task	Jumlah VM	Pengulangan
S1	Synthetic DAG	100	10	10
S2	Synthetic DAG	100	20	10
S3	Synthetic DAG	100	40	10
S4	Synthetic DAG	250	10	10
S5	Synthetic DAG	250	20	10
S6	Synthetic DAG	250	40	10
S7	Synthetic DAG	500	10	10
S8	Synthetic DAG	500	20	10
S9	Synthetic DAG	500	40	10
S10	Montage	≈ 100	10	10*
S11	Montage	≈ 100	20	10*
S12	Montage	≈ 100	40	10*
S13	CyberShake	≈ 100	10	10*
S14	CyberShake	≈ 100	20	10*
S15	CyberShake	≈ 100	40	10*

Catatan: pada DAG sintetis, 10 pengulangan menggunakan 10 DAG berbeda (seed acak) dan hasilnya dilaporkan sebagai rata-rata dan simpangan baku. (*) pada Montage dan CyberShake, struktur DAG tetap sehingga pengulangan dipakai untuk mengukur variasi scheduling runtime.

Output yang dikumpulkan pada setiap skenario adalah makespan, total konsumsi energi, utilisasi CPU, utilisasi memori, dan scheduling runtime. Hasil setiap skenario dibandingkan untuk mengetahui pengaruh ukuran workload dan jumlah VM terhadap kinerja penjadwalan. Konfigurasi datacenter dan host sama untuk seluruh skenario (Tabel 3.5), sedangkan komposisi VM mengikuti Tabel 3.7.

4.2 Estimasi Analitis Kinerja HEFT pada Skenario S1

Sebelum simulator diimplementasikan, dilakukan estimasi analitis pada skenario S1 (DAG sintetis 100 task pada 10 VM) untuk memperoleh gambaran orde besar kinerja HEFT dan sebagai pembanding hasil simulasi. Estimasi ini bersifat perkiraan berdasarkan nilai rata-rata parameter, bukan hasil simulasi.

4.2.1 Asumsi

Tabel 4.2 Asumsi estimasi analitis skenario S1

Parameter	Nilai
Jumlah task	100 task
Rata-rata panjang task	5.500 MI (total beban = 550.000 MI)
Jumlah level DAG	≈ 10 level (rata-rata lebar ≈ 10 task)
Komposisi VM	2 S, 3 M, 3 L, 2 XL (25 vCPU; 39.000 MIPS)
Rata-rata MIPS per vCPU antar-VM	1.425 MIPS
Kecepatan per vCPU tercepat	2.000 MIPS (tipe XL)
Rata-rata komunikasi antar-task ($\bar{c}$)	≈ 1,9 s (CCR ≈ 0,5)
Fraksi komunikasi yang dihilangkan HEFT (task sesama VM)	≈ 50%
Panjang task pada lintasan kritis	≈ 7.000 MI (lebih panjang dari rata-rata)
Daya host	ΣP_{idle} = 980 W; $\Sigma (P_{max} - P_{idle})$ = 1.360 W

4.2.2 Estimasi Makespan

Langkah pertama adalah menghitung dua batas bawah makespan. Batas bawah berdasarkan beban (*work bound*) diperoleh dari total beban dibagi total kapasitas VM:

$$LB_{work} = \frac{L_i}{vcpu_v \cdot MIPS_v} = \frac{550.000}{39.000} \approx 14,1\ s \quad (4.1)$$

Batas bawah berdasarkan lintasan kritis (*critical path bound*) dihitung dengan 10 task berurutan berpanjang rata-rata 7.000 MI yang dijalankan pada vCPU tercepat tanpa komunikasi:

$$CP_{MIN} = \frac{10 \times 7.000\ MI}{2.000\ MIPS} = 35\ s \quad (4.2)$$

Batas bawah beban (14,1 s) jauh lebih kecil daripada batas bawah lintasan kritis (35 s). Artinya makespan pada S1 didominasi oleh struktur ketergantungan, bukan

oleh kekurangan kapasitas. Rata-rata biaya komputasi task adalah $\bar{w}$ = 5.500 / 1.425 ≈ 3,86 s sehingga rata-rata biaya komunikasi adalah $\bar{c}$ = CCR × $\bar{w}$ ≈ 1,9 s. Pada lintasan kritis terdapat 9 sisi; dengan sekitar setengah dari komunikasi tersebut dihilangkan oleh penempatan task pada VM yang sama, tambahan waktu komunikasi adalah:

$$T_{comm} \approx 9 \times 1,9\,s \times 0,5 \approx 8,7\,s \quad (4.3)$$

Sehingga estimasi makespan HEFT pada S1 adalah sekitar 35 + 8,7 ≈ 44 s. Dengan tambahan antrean akibat persaingan sumber daya (diasumsikan sekitar 10%), makespan diperkirakan berada pada rentang 44–48 s. SLR yang diharapkan adalah 44/35 hingga 48/35, yaitu sekitar 1,25–1,37.

4.2.3 Estimasi Energi dan Utilisasi

Dengan makespan sekitar 46 s (nilai tengah), energi dibagi menjadi komponen idle dan dinamis. Komponen idle adalah 980 W × 46 s ≈ 45,1 kJ. Untuk komponen dinamis, diasumsikan HEFT memilih VM cepat sehingga kecepatan efektif rata-rata sekitar 1.700 MIPS per vCPU; beban total setara 100 × 5.500 / 1.700 ≈ 324 vCPU-detik. Utilisasi CPU host adalah 324 / (128 × 46) ≈ 5,5%, sehingga energi dinamis adalah 1.360 W × 0,055 × 46 s ≈ 3,4 kJ. Utilisasi CPU terhadap kapasitas vCPU VM adalah 324 / (25 × 46) ≈ 28%.

Estimasi runtime penjadwalan berdasarkan kompleksitas O(e × p): dengan sekitar 180 sisi dan 10 VM, diperlukan sekitar 1.800 evaluasi EFT ditambah pengurutan 100 task. Jumlah operasi tersebut sangat kecil, sehingga runtime diperkirakan berorde beberapa milidetik (di luar waktu inisialisasi simulator). Ringkasan estimasi ditunjukkan pada Tabel 4.3.

Tabel 4.3 Ringkasan estimasi analitis skenario S1

Metrik	Estimasi	Keterangan
Batas bawah beban	≈ 14,1 s	Persamaan (4.1)
Batas bawah lintasan kritis (CP_{MIN})	35 s	Persamaan (4.2)
Makespan HEFT	≈ 44–48 s	CP + komunikasi + antrean
SLR	≈ 1,25–1,37	Makespan / CP_{MIN}
Energi total	≈ 48,5 kJ (≈ 0,0135 kWh)	Idle ≈ 45,1 kJ; dinamis ≈ 3,4 kJ

Metrik	Estimasi	Keterangan
Utilisasi CPU host	≈ 5,5%	324 vCPU-s / (128 PE × 46 s)
Utilisasi CPU vCPU VM	≈ 28%	324 vCPU-s / (25 vCPU × 46 s)
Scheduling runtime	Beberapa milidetik	O(e × p), e ≈ 180, p = 10

4.3Analisis Awal

Estimasi pada Subbab 4.2 menghasilkan beberapa temuan awal yang akan diuji pada tahap simulasi.

Pertama, makespan didominasi lintasan kritis. Pada S1, batas bawah lintasan kritis (35 s) lebih dari dua kali batas bawah beban (14,1 s). Penambahan VM hanya mengurangi antrean, bukan panjang lintasan kritis. Oleh karena itu, peningkatan jumlah VM dari 10 ke 20 dan 40 diperkirakan memberi penurunan makespan yang semakin kecil (*diminishing returns*), sedangkan pada workflow yang lebih lebar seperti Montage penambahan VM diperkirakan lebih bermanfaat.

Kedua, energi didominasi daya idle. Pada S1 komponen idle sekitar 93% dari energi total. Hal ini disebabkan seluruh host diasumsikan tetap menyala selama simulasi dan utilisasi CPU host rendah. Konsekuensinya, pada rancangan ini pengurangan energi terutama dicapai melalui pemendekan makespan, sebab makespan menentukan lama daya idle dibayarkan. Penambahan VM tidak menambah daya idle karena jumlah host tetap delapan, tetapi menurunkan utilisasi. HEFT yang mengejar EFT terkecil bertindak selaras dengan tujuan energi pada rancangan ini, namun tidak memanfaatkan peluang menghemat energi dari penempatan task pada host yang lebih efisien atau mematikan host yang tidak dipakai.

Ketiga, utilisasi menurun bila VM ditambah. Beban total tetap sedangkan kapasitas bertambah, sehingga utilisasi CPU dan RAM turun ketika VM meningkat dari 10 ke 40. Sebaliknya, ketika jumlah task meningkat (100 → 500) pada jumlah VM tetap, utilisasi cenderung naik.

Keempat, deadline. Bila $D = \beta \times CP_{MIN}$ dengan $\beta = 1{,}5$, maka D = 52,5 s pada S1. Estimasi makespan 44–48 s masih memenuhi batas tersebut, tetapi

margin tipis; pada β = 2,0 (D = 70 s) batas terpenuhi dengan longgar. Tren prediksi antar-skenario dirangkum pada Tabel 4.4.

Tabel 4.4 Prediksi tren kinerja HEFT antar-skenario

Perubahan kondisi	Makespan	Energi	Utilisasi CPU
Jumlah VM 10 → 20 → 40	Turun, makin kecil	Turun tipis	Turun
Jumlah task 100 → 250 → 500	Naik	Naik	Naik
Montage vs sintetis	Manfaat VM lebih besar	Mengikuti makespan	Variatif
CyberShake vs sintetis	Sensitif pada komunikasi	Mengikuti makespan	Variatif
Scheduling runtime saat n naik	Naik mendekati kuadratik	—	—

4.4 Ekspektasi Hasil Simulasi

Berdasarkan estimasi analitis, hasil simulasi HEFT diharapkan menunjukkan hal-hal berikut.

- **Makespan**: pada S1, nilai simulasi berada pada orde 40–55 s dan SLR berada antara 1,2 dan 1,5. Makespan menurun seiring bertambahnya VM dengan laju yang semakin kecil.
- **Energi**: didominasi daya idle host (sekitar 90% atau lebih), sehingga tren energi mengikuti tren makespan.
- **Utilisasi**: utilisasi CPU host rendah (di bawah 10% pada S1) dan turun ketika VM bertambah; utilisasi CPU terhadap vCPU VM sekitar 25–35% pada S1.
- **Scheduling runtime**: sangat kecil (orde milidetik) untuk seluruh ukuran workflow, dan naik lebih cepat dari linear terhadap jumlah task.
- **Deadline**: pada β = 2,0 hampir seluruh skenario memenuhi deadline; pada β = 1,5 skenario dengan komunikasi tinggi berpotensi melanggar.

Estimasi dianggap konsisten dengan simulasi apabila deviasi antara nilai estimasi dan rata-rata hasil simulasi pada S1 tidak lebih dari 25% untuk makespan dan energi. Apabila deviasi lebih besar, asumsi pada Tabel 4.2 (misalnya fraksi

komunikasi yang dihilangkan atau panjang task pada lintasan kritis) akan ditinjau ulang.

4.5 Catatan Teknis dan Risiko Implementasi

Beberapa catatan teknis dan risiko yang perlu diantisipasi pada tahap implementasi ditunjukkan pada Tabel 4.5.

Tabel 4.5 Risiko implementasi dan rencana mitigasi

Risiko / catatan teknis	Dampak	Mitigasi
Pemodelan energi host belum tersedia bawaan pada WorkflowSim	Metrik energi tidak dapat dihitung	Mengintegrasikan paket power CloudSim (PowerHost, model linear) atau menghitung energi dari log utilisasi
Heterogenitas VM (MIPS berbeda) pada model HEFT	Biaya komputasi tidak terhitung benar bila MIPS tidak diatur	Mengatur MIPS setiap VM sesuai Tabel 3.6 dan memvalidasi $w_{i,v}$ secara manual pada DAG kecil
Biaya komunikasi antar-VM/antar-datacenter	CCR aktual menyimpang dari rencana	Memverifikasi CCR aktual setelah DAG dibangkitkan dan menyesuaikan ukuran data
Ketersediaan berkas DAX Montage/CyberShake ukuran ≈ 100 task	Ukuran workflow berbeda dari rencana	Menggunakan varian terdekat yang tersedia dan mencatat jumlah task aktual
Kapasitas host tidak cukup pada 40 VM	VM gagal dialokasikan	Total kebutuhan 100 vCPU sudah di bawah 128 PE; pengecekan alokasi sebelum simulasi
Variasi hasil akibat DAG acak	Kesimpulan tidak stabil	10 pengulangan dengan seed berbeda; laporkan rata-rata dan simpangan baku
Pengukuran runtime dipengaruhi JVM (*warm-up*)	Runtime tidak representatif	Mengabaikan proses pemanasan dan mengambil rata-rata beberapa kali jalan

BAB V PENUTUP

5.1 Kesimpulan Sementara

Berdasarkan draft desain awal yang telah disusun, dapat disimpulkan hal-hal berikut.

1. Workload yang dipilih adalah workflow DAG yang terdiri atas DAG sintetis (100, 250, dan 500 task) untuk menguji skalabilitas dan workflow dunia nyata Montage dan CyberShake untuk menguji struktur ketergantungan yang realistis.
2. Arsitektur cloud yang dirancang terdiri atas 2 datacenter heterogen, 8 host fisik (4 host per datacenter), 10/20/40 VM dengan 1–4 vCPU dan 2–8 GB RAM, serta task/cloudlet yang berasal dari workflow DAG, dan disimulasikan menggunakan WorkflowSim/CloudSim.
3. Fungsi objektif yang dioptimalkan dan dievaluasi adalah minimasi makespan dan minimasi konsumsi energi. HEFT digunakan untuk menghasilkan pemetaan task ke VM, dan hasilnya dievaluasi terhadap kedua objektif.
4. Metrik yang diukur meliputi makespan, konsumsi energi, utilisasi CPU dan RAM, serta scheduling runtime, dengan batasan cloud berupa ketergantungan, kapasitas sumber daya, ketersediaan VM, dan deadline opsional.
5. Estimasi analitis pada skenario S1 menunjukkan makespan HEFT sekitar 44–48 s yang didominasi lintasan kritis dan energi sekitar 48,5 kJ yang didominasi daya idle host. Estimasi ini masih perlu diverifikasi melalui simulasi.

5.2 Rencana Tindak Lanjut

Rencana tindak lanjut pada tahap implementasi (Tugas 4) adalah sebagai berikut.

1. Menyiapkan lingkungan WorkflowSim/CloudSim dan membangun konfigurasi datacenter, host, dan VM sesuai Tabel 3.4 sampai Tabel 3.7.

2. Membangkitkan DAG sintetis sesuai Tabel 3.2 dan menyiapkan berkas DAX Montage dan CyberShake.
3. Mengimplementasikan atau mengonfigurasi penjadwal HEFT (perhitungan $rank_u$ dan pemilihan VM berdasarkan EFT) serta memvalidasi hasilnya pada DAG kecil.
4. Menambahkan perhitungan energi berbasis model daya linear dan pencatatan metrik utilisasi serta scheduling runtime.
5. Menjalankan 15 skenario pengujian (S1–S15) dengan pengulangan, mengumpulkan metrik, dan membandingkannya dengan estimasi analitis pada Subbab 4.2.
6. Menggunakan hasil HEFT sebagai baseline untuk pengembangan pendekatan penjadwalan lanjutan, misalnya pendekatan yang memperhitungkan energi secara eksplisit atau pendekatan multiobjektif.

DAFTAR PUSTAKA

[1]P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” NIST Special Publication 800-145, National Institute of Standards and Technology, 2011.

[2]M. Armbrust *et al.*, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[3]J. D. Ullman, “NP-complete scheduling problems,” *Journal of Computer and System Sciences*, vol. 10, no. 3, pp. 384–393, 1975.

[4]H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.

[5]R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, “CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms,” *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.

[6]W. Chen and E. Deelman, “WorkflowSim: A toolkit for simulating scientific workflows in distributed environments,” in *Proc. IEEE 8th International Conference on E-Science*, 2012.

[7]E. Deelman *et al.*, “Pegasus, a workflow management system for science automation,” *Future Generation Computer Systems*, vol. 46, pp. 17–35, 2015.

[8]G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, “Characterizing and profiling scientific workflows,” *Future Generation Computer Systems*, vol. 29, no. 3, pp. 682–692, 2013.

[9]S. Bharathi *et al.*, “Characterization of scientific workflows,” in *Proc. 3rd Workshop on Workflows in Support of Large-Scale Science (WORKS)*, 2008.

[10]A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for Cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012.

[11]X. Fan, W.-D. Weber, and L. A. Barroso, “Power provisioning for a warehouse-sized computer,” in *Proc. 34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, pp. 13–23.